

1. Student Details

- **Name:** Piyush Maharana
- **Roll Number:** 24f2006489
- **Email:** 24f2006489@ds.study.iitm.ac.in
- **About Me:** I am a student in the IIT Madras BS Degree program with a strong focus on Full-Stack Web Development. I enjoy building highly reactive, enterprise-grade applications using frameworks like Vue.js and Flask, and optimizing them with modern data structures and system architecture. I prefer calling myself ‘jack of all trades but master of none’. But if we dig deeper into this, it says “Jack of all trades but master of none, but oftentimes better than master of one”.

2. Project Details

- **Project Title:** Trekking Management Application (TMA) - V2
- **Problem Statement:**
Adventure organizations currently rely on manual coordination and spreadsheets, making it difficult to efficiently manage trek approvals, slot availability, and participant tracking. This project aims to build a centralized, role-based web application to streamline these operations for admins, staff, and trekkers.
- **Approach:**
I developed a decoupled web application using Flask to serve RESTful APIs and Vue.js to build a highly reactive, single-page frontend. The architecture implements a unified user model with strict Role-Based Access Control (RBAC) via JWTs. To ensure high performance and handle concurrency, I integrated Redis for caching API responses and Celery for asynchronous background tasks like sending automated emails and generating CSV exports.

3. AI/LLM Declaration

I used an LLM (Gemini) to assist with architectural guidance, specifically for understanding advanced integrations like Server-Sent Events (SSE) for real-time reactivity, configuring Celery for asynchronous background tasks, and debugging state management in Vue.js. All final implementation logic, debugging, and system integration were executed and verified manually.

Component / Tool	Purpose / Role in Project	Usage %	Notes
Flask (App + API)	Backend REST API logic, routing, and RBAC implementation	15%	Assisted with JWT token integration and security gate logic.

Vue.js / UI	Reactive frontend, component structure, and state management	20%	Used for debugging component scoping and modal "Smart Payload" logic.
Redis / SSE / Celery	Caching, real-time sync, async background workers	25%	Guided the SSE "Radio Antenna" implementation and Celery beat scheduling.
SQLite + SQLAlchemy	Data modeling and database constraints	5%	Minor syntax checks for SQLAlchemy model relationships.
Bootstrap / CSS	UI styling and responsive layouts	10%	Assisted with Neumorphic/Glassmorphic CSS properties.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework used to build the RESTful API.
Vue.js	Frontend framework used to build the reactive Single-Page Application (SPA).
SQLAlchemy	Object Relational Mapper (ORM) used for database modeling and queries.
SQLite	Lightweight local database for storing users, treks, and bookings data.
Redis	In-memory data store used for caching API responses and as a message broker for background jobs.
Celery	Asynchronous task queue for handling scheduled jobs (reports/reminders) and user-triggered jobs (CSV exports).
Flask-JWT-Extended	Used for secure, token-based authentication and Role-Based Access Control (RBAC).
Flask-SSE	Used to implement Server-Sent Events for real-time dashboard synchronization without page refreshes.

Bootstrap & Custom CSS	Used for responsive HTML layout generation and custom Neumorphic/Glassmorphic styling.
Chart.js (vue-chartjs)	Used to visualize platform analytics and user demographics on the Admin dashboard.

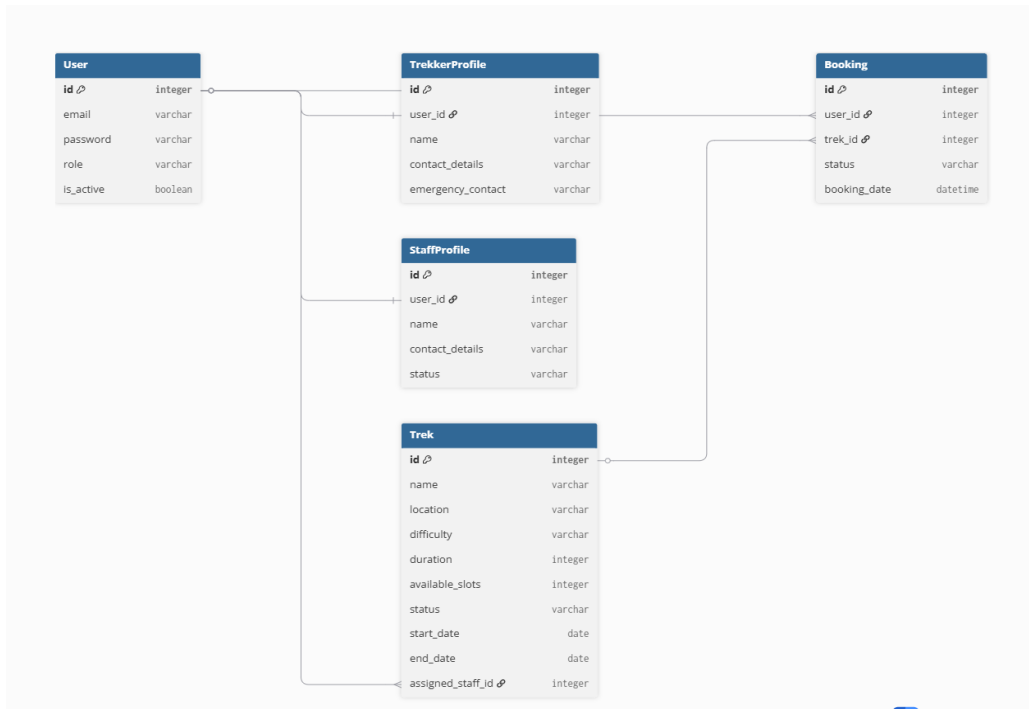
5. Database Schema / ER Diagram

Tables & Key Attributes:

- **User:** Core authentication table (id, email, password, role, is_active).
- **TrekkerProfile:** Details for registered trekkers (id, user_id, name, contact_details, emergency_contact).
- **StaffProfile:** Details for staff members (id, user_id, name, contact_details, status).
- **Trek:** Stores expedition details (id, name, location, difficulty, duration, available_slots, status, start_date, end_date, assigned_staff_id).
- **Booking:** Bridge table recording user reservations (id, user_id, trek_id, status, booking_date).

Relationships:

- **One-to-One:** User ↔ TrekkerProfile (Each trekker has one profile).
- **One-to-One:** User ↔ StaffProfile (Each staff has one profile).
- **One-to-Many:** User (Staff) ↔ Trek (One staff member can be assigned to multiple treks over time).
- **One-to-Many:** User (Trekker) ↔ Booking (One user can have multiple bookings).
- **One-to-Many:** Trek ↔ Booking (One trek can have multiple bookings from different users).



6. API Resource Endpoints

Endpoint	Method	Description
/login	POST	Authenticates the user, verifies credentials, and returns a JWT access token along with the user role.
/register	POST	Registers a new Trekker, hashes the password, and creates an associated TrekkerProfile.
/admin/dashboard/stats	GET	Fetches aggregate statistics (total trekkers, staff, treks, bookings) for the admin overview, optimized with Redis caching.
/admin/staff	POST	Creates a new staff member account and profile (restricted to Admin).
/admin/trek	POST	Creates a new trekking expedition and handles initial staff assignment status.
/admin/trek/<id>/assign	PUT	Assigns a staff member to a trek while actively verifying the 10-day scheduling buffer constraint.

/treks/available	GET	Fetches a list of open and approved treks for the Trekker marketplace, supporting location, difficulty, and duration filters (Redis cached).
/book	POST	Processes a booking, decrements available slots, checks for date overlaps, and triggers real-time SSE broadcasts.
/staff/trek/<id>/status	PUT	Allows assigned staff members to update the current phase/status of a trek (e.g., Open, Completed).
/trekker/export-history	POST	Triggers an asynchronous Celery task to generate a CSV export of the user's personal booking ledger.

8. Video Presentation

Drive Link:

https://drive.google.com/file/d/1Njg62fcUsn7KEcltV7LQ5KnvMI3qoA8h/view?usp=drive_link